BÖLÜM 10 / MODULE 10

GitHub Actions CI/CD & Büyük Mezuniyet Senaryosu

GitHub Actions CI/CD & The Grand Enterprise Finale

Uçtan uca otomasyon fabrikası: GitHub Actions iş akışları, Spring Boot Maven testleri, Trivy güvenlik taramaları, GitOps köprüsü ve büyük mezuniyet projesi.

The end-to-end automation factory: GitHub Actions workflows, Spring Boot Maven builds, Trivy CVE scanning, bridging CI to GitOps, and the grand live deployment finale.

55

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

GitHub Actions Temelleri: Workflows, Jobs, Steps ve Runners

GitHub Actions Fundamentals: Workflows, Jobs, Steps, and Runners

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kodunuzu GitHub'a `git push` yaptığınız anda harekete geçen görünmez bir fabrika düşünün! 1) Workflow: Fabrikanın üretim planıdır (`.github/workflows/ci.yaml`). 2) Trigger (Tetikleyici): Zilin çalmasıdır ('Main branch'e kod geldi!'). 3) Jobs (İşler): Fabrikanın farklı departmanlarıdır ('Biri test yapsın, diğeri Docker derlesin'). 4) Runners: Bu işleri yapan bulut bilgisayarlarıdır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine an invisible, robotic smart factory that fires up the millisecond you run `git push`! 1) Workflow: The factory blueprint (`.github/workflows/ci.yaml`). 2) Trigger (Event): The starting bell ringing ('Code pushed to main!'). 3) Jobs: Distinct assembly lines ('Line 1 tests Java, Line 2 bakes the container'). 4) Runners: The virtual machines in the cloud executing the steps!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

GitHub Actions mimarisinin yapı taşları: 1) `Events`: İş akışını başlatan olaylar (`push`, `pull_request`, `schedule`, `workflow_dispatch`). 2) `Jobs`: Varsayılan olarak paralel çalışan bağımsız görev bloklarıdır (`needs: [build]` ile sıralı bağımlılık kurulur). 3) `Steps`: Sırayla çalıştırılan tekil kabuk komutları (`run`) veya hazır topluluk eylemleri (`uses: actions/checkout@v4`). 4) `Runners`: Ubuntu, macOS veya Windows barındıran geçici sanal makinelerdir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The core taxonomy of GitHub Actions: 1) `Events` trigger execution (`push`, `pull_request`, webhooks). 2) `Jobs` run concurrently by default on isolated compute runners unless sequenced via `needs: [jobA]`. 3) `Steps` execute sequentially inside a job environment as shell scripts (`run`) or composite action modules (`uses`). 4) `Runners` are ephemeral hosted virtual machines or self-hosted servers.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# .github/workflows/hello-ci.yaml
name: First Spring Boot Workflow
on:
  push:
    branches: [ "main" ]

jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - name: Kodu Klonla
        uses: actions/checkout@v4

      - name: Terminale Yazdır
        run: |
          echo "🚀 Java Spring Boot CI Pipeline Başlatıldı!"
          echo "Commit SHA: ${{ github.sha }}"
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir iş akışındaki adımlar (steps) aynı sanal makinede ardışık çalışır ve dosya sistemini paylaşır. Ancak farklı 'Job'lar farklı sanal makinelerde çalıştığı için aralarında dosya paylaşımı yapmak istiyorsanız `actions/upload-artifact` kullanmalısınız!

🇬🇧 WATCH OUT

Steps within a job share the runner filesystem; distinct Jobs run on separate virtual machines! Pass compiled artifacts between jobs using `upload-artifact` and `download-artifact`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- GitHub Actions harici CI sunucusu (Jenkins) kurma ihtiyacını ortadan kaldırır.
- YAML tabanlı bildirici sözdizimi doğrudan projenin Git reposunda saklanır.
- Geniş GitHub Marketplace ekosistemi hazır otomasyon eylemleri sunar.

- Eliminates the maintenance overhead of managing dedicated external CI/CD servers.
- Declarative workflow definitions live directly inside your project repository.
- Leverages a vast marketplace of battle-tested open-source actions for rapid setup.

56

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Spring Boot CI Pipeline: Maven Test, Build ve Kalite Kontrolü

Spring Boot CI Pipeline: Maven Test, Build & Quality Gates

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Hatalı bir kodun üretim hattına geçmesini engelleyen acımasız kapı bekçisi! Yazılımcı gece yarısı dalgınlıkla bir hataya sebep olduysa veya birim testleri (Unit Tests) bozduysa, GitHub Actions o kodu anında durdurur: Kırmızı çarpı işaretini basar, yöneticilere haber verir ve bozuk kodun Docker imajı olmasına ASLA izin vermez!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The unyielding quality gatekeeper! If a sleepy engineer accidentally introduces a regression or breaks a unit test, GitHub Actions slams the brakes on the assembly line: it raises a red flag, alerts the engineering team, and strictly prevents the broken build from ever producing a container image!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Etkili bir Java CI adımı şunları içerir: 1) `actions/setup-java@v4`: Eclipse Temurin JDK 21 kurulumu ve `cache: 'maven'` ile `~/.m2` deposunun önbelleğe alınması (derleme süresini 4 dakikadan 30 saniyeye düşürür). 2) `mvn clean test`: Tüm JUnit 5 ve Spring Boot MockMvc testlerini çalıştırır. 3) Test raporlarının ve kod kapsama oranlarının (JaCoCo) üretilmesi. 4) `mvn package -DskipTests`: Üretim JAR dosyasının derlenmesi.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A resilient Spring Boot CI pipeline includes: 1) `setup-java` configuring JDK 21 with automatic Maven dependency caching (shrinking build times from minutes to seconds). 2) `mvn clean test` executing JUnit 5 unit and integration test suites. 3) JaCoCo test coverage enforcement. 4) Packaging the final layered production executable JAR via Spring Boot Maven plugin.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# Maven CI Job Bloğu:
jobs:
  test-and-package:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: JDK 21 Temurin Kur ve Maven Önbellekle
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - name: Birim ve Entegrasyon Testlerini Çalıştır
        run: mvn clean test

      - name: Üretim JAR Paketini Derle
        run: mvn package -DskipTests
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Maven önbellekleme (`cache: maven`) yapmazsanız, GitHub Actions her commit'te tüm Spring Boot kütüphanelerini internetten sıfırdan indirir; bu da dakikalarca zaman kaybına ve GitHub Actions dakikalarınızın hızla tükenmesine yol açar.

🇬🇧 WATCH OUT

Omitting Maven dependency caching forces the runner to re-download hundreds of megabytes of dependencies on every push, wasting pipeline minutes and developer time.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Otomatik testler bozuk kodların müşterilere ulaşmasını %100 engeller.
- Maven önbellekleme derleme sürelerini kat kat hızlandırır.
- Test raporları her Pull Request üzerinde şeffaf biçimde incelenebilir.

- Automated CI testing guarantees that failing code cannot slip unnoticed into production.
- Maven dependency caching slashes build times from minutes down to seconds.
- Test outcomes are surfaced directly on GitHub Pull Requests for peer review.

57

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Docker İmajı Paketleme, Trivy Güvenlik Taraması ve GHCR Push

Docker Packaging, Trivy Vulnerability Scanning & GHCR Push

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Fabrikadan çıkan Spring Boot uygulamasını çelik bir kargo konteynerine koyuyoruz. Ama hemen gemiye yüklemiyoruz! Önce konteyneri X-Ray cihazından (Trivy güvenlik tarayıcısı) geçiriyoruz. İçinde tehlikeli bir virüs veya güvenlik açığı (CVE) var mı? Temiz raporu çıkarsa, GitHub Container Registry'ye (GHCR) resmi şirket mührüyle teslim ediyoruz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

We pack our freshly compiled Spring Boot binary into a standard steel container. But we don't dispatch it blindly! First, it passes through a high-tech airport X-Ray scanner (Trivy vulnerability scanner). Are there any high-severity CVEs? Only when it receives a clean bill of health do we push it with an immutable cryptographic tag to GHCR!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bu aşama üç kritik adımdan oluşur: 1) `docker/setup-buildx-action`: Docker Buildx ile hızlı, çok katmanlı önbellek kullanan derleme altyapısı kurar. 2) `aquasecurity/trivy-action`: Üretilen konteyner imajını işletim sistemi paketleri ve Java JAR bağımlılıklarındaki (Log4Shell vb.) bilinen CVE açıkları için tarar; `severity: 'CRITICAL'` bulunursa pipeline'ı keser (`exit-code: 1`). 3) `ghcr.io`: GitHub'ın yerleşik OCI uyumlu konteyner deposuna güvenli push yapılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

This stage executes: 1) Docker Buildx initialization for advanced multi-platform and layer caching. 2) Trivy vulnerability analysis scanning both OS packages and application dependencies against national vulnerability databases; fails the build on Critical CVE thresholds. 3) Authenticated pushing to GitHub Container Registry (`ghcr.io`) using short-lived runner permissions.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Docker Derleme, Tarama ve GHCR Push:
- name: GitHub Container Registry Girişi
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Docker İmajını Derle
  uses: docker/build-push-action@v5
  with:
    context: .
    load: true
    tags: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}

- name: Trivy ile Güvenlik Taraması Yap
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}'
    severity: 'CRITICAL,HIGH'
    exit-code: '0' # Kritik açıkları raporla

- name: İmajı Registry'ye Gönder
  run: docker push ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İmaj etiketi olarak ASLA `:latest` kullanmayın! `:latest` etiketi hangi kodun canlıda çalıştığını anlamayı imkansız kılar ve K8s imaj önbelleği yüzünden güncellemelerin çekilmemesine neden olur. Her zaman git commit SHA'sını (`: ${{ github.sha }}`) kullanın.

🇬🇧 WATCH OUT

NEVER tag images with `:latest` in production pipelines! `:latest` obscures version provenance and defeats Kubernetes image caching. Always tag with the immutable Git commit SHA.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Trivy güvenlik açığı olan paketlerin üretim ortamına sızmasını önler.
- GHCR doğrudan GitHub ekosistemiyle entegre, yüksek hızlı imaj barındırır.
- Her imaj benzersiz Git SHA etiketiyle mühürlenerek tam izlenebilirlik sağlanır.

- Automated Trivy scanning blocks known vulnerable dependencies from reaching production.
- GHCR offers seamless authentication and high-bandwidth image distribution.
- Tagging with Git commit SHA guarantees 100% traceability between code and containers.

58

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

CI ile GitOps Köprüsü: GitHub Actions'ın Flux Deposunu Güncellemesi

Bridging CI to GitOps: GitHub Actions Updating Flux Manifests

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

İki muazzam dünyayı birleştiren altın köprü! GitHub Actions (CI) derlemeyi bitirdiğinde doğrudan gidip K8s kümesine saldırmaz (Bu GitOps kuralına aykırıdır). Bunun yerine gider, Flux'ın izlediği GitOps filosunun deposundaki YAML dosyasını açar: 'İmaj etiketini az önce derlediğim yeni SHA ile güncelledim' diyerek bir commit atar. Ve sahneden çekilir; gerisini Flux halleder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Bridge connecting two mighty engineering realms! When GitHub Actions (CI) finishes building, it NEVER attacks the Kubernetes cluster directly (that violates GitOps!). Instead, it walks over to the separate GitOps repository tracked by Flux, updates the image tag inside `deployment.yaml` with the fresh commit SHA, and pushes a clean commit. CI exits; Flux takes the wheel!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kurumsal ayrım prensibi (Separation of Concerns): 1) Kod Reposu (Uygulama kodu + CI pipeline). 2) GitOps Reposu (K8s manifestoları + Ortam ayarları). GitHub Actions derlemeyi bitirdiğinde: `actions/checkout` ile GitOps reposunu klonlar, `clusters/production/.../deployment.yaml` içindeki imaj etiketini yeni derlenen `image: ghcr.io/...:${{ github.sha }}` ile günceller (sed veya Kustomize edit ile), ve GitOps reposuna bir 'chore(release)' commit'i atar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Separation of concerns: 1) App Repo (source code + test pipelines), 2) GitOps Fleet Repo (Kubernetes manifests + infrastructure state). At the tail of the CI run, the workflow clones the fleet repo, updates the target deployment manifest with the newly pushed container SHA via `kustomize edit set image` or `sed`, and commits back to the fleet repo, triggering Flux in-cluster.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# GitHub Actions'tan GitOps Reposuna Otomatik Terfi (Promotion):
- name: GitOps Deposunu Klonla
  uses: actions/checkout@v4
  with:
    repository: 'my-org/k8s-gitops-fleet'
    token: ${{ secrets.GITOPS_DEPLOY_PAT }}
    path: fleet-repo

- name: İmaj Etiketini Güncelle ve Push Et
  run: |
    cd fleet-repo/clusters/production/coffee-app
    sed -i "s|image: .*|image: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}|
g" deployment.yaml
    git config user.name "github-actions[bot]"
    git config user.email "actions@github.com"
    git add deployment.yaml
    git commit -m "chore(release): promote coffee-app to version ${{ github.sha }}"
    git push origin main
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama kodunun olduğu repo ile GitOps manifestolarının olduğu repoyu ayrı tutmak en iyi kurumsal pratiktir. Böylece her Java kod değişikliğinde sonsuz bir CI/CD döngüsü (infinite trigger loop) yaşanması engellenir.

🇬🇧 WATCH OUT

Decoupling application repositories from GitOps fleet repositories prevents catastrophic infinite pipeline trigger recursion loops.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CI altyapısı hiçbir zaman doğrudan Kubernetes kümesine bağlanmaz.
- Dağıtım tetikleyicisi Git üzerindeki temiz bir commit'e dönüşür.
- Tek bir `git revert` komutuyla tüm üretim kümesi önceki sürüme geri döner.

- CI build runners require zero direct network access or credentials to production clusters.
- The release event is formalized as a transparent Git commit in the fleet repo.
- Rolling back production is as simple and safe as executing `git revert` on the fleet repo.

59

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Çoklu Ortam Dağıtımı (Dev -> Staging -> Prod) ve Canary

Multi-Stage Environment Promotions: Dev, Staging & Production

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Yeni geliştirilen bir ilacı önce laboratuvarda fare üzerinde (Dev ortamı), sonra gönüllü küçük bir grupta (Staging ortamı), her şey yolundaysa tüm dünyada (Production ortamı) kullanıma sunarsınız! Hatta üretimde bile önce müşterilerin sadece %10'una yeni versiyonu gösteren 'Kanarya' (Canary) testi yaparsınız. Bir hata varsa sadece %10 etkilenir, anında geri çekilir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't test a brand-new experimental medicine on the entire global population on day one! You test in the lab (Dev), validate with a small volunteer trial (Staging), and only then release to the public (Production)! And even in production, you use a 'Canary' release: route 10% of users to the new version first. If error rates tick up, kill it instantly before 90% notice!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kustomize 'Overlays' mimarisi ile ortam yönetimi: `base/` (Ortak Deployment ve Service tanımları) üzerine `overlays/dev/` (1 kopya, düşük CPU), `overlays/staging/` ve `overlays/production/` (10 kopya, yüksek CPU, HPA, katı kısıtlar) bindirilir. GitHub Actions 'Environments' mekanizmasıyla Production ortamına geçiş için kıdemli mühendis onayı (Manual Approval Gate) zorunlu kılınabilir. Flagger ile Canary analizleri (HTTP hata oranı <%1) yürütülür.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Multi-tier promotion leverages Kustomize Overlays: a common `base/` manifest stack is patched by `overlays/dev/`, `overlays/staging/`, and `overlays/production/` (varying replicas, resource limits, and secrets). GitHub Environments enforce protection rules requiring manual engineering lead sign-offs before staging can promote to production.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Kustomize Ortam Ağacı:
# apps/coffee-app/
# ├── base/
# │   ├── deployment.yaml
# │   └── service.yaml
# └── overlays/
#     ├── dev/
#     │   └── kustomization.yaml (replicas: 1)
#     └── production/
#         └── kustomization.yaml (replicas: 5, HPA aktif)

# Production Terfisi İçin GitHub Actions Onay Adımı:
# environment:
#   name: production
#   url: https://coffee.production.company.com
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Farklı ortamlar için ayrı ayrı devasa YAML kopyaları oluşturmayın (Kopya-Yapıştır kirliliği!). Ortak şablonu `base/` altında tutun ve sadece değişen ayarları `overlays/` içinde yamalayın (DRY - Don't Repeat Yourself).

🇬🇧 WATCH OUT

Never duplicate massive YAML manifests across environments! Maintain shared definitions in `base/` and patch environment deltas inside `overlays/`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Kustomize overlays tek bir temel şablonla farklı ortamları yönetir.
- GitHub Actions Onay Kapıları (Approval Gates) kontrolsüz prod geçişlerini durdurur.
- Kanarya (Canary) dağıtımları hata anında felaketi önceden yakalar.

- Kustomize overlays eliminate manifest duplication while supporting environment nuances.
- Manual approval gates prevent unverified releases from hitting production.
- Canary deployments limit blast radius to a microscopic fraction of live traffic.

60

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

BÜYÜK MEZUNİYET FİNALİ: Uçtan Uca Kurumsal Spring Boot Akışı!

THE GRAND FINALE: Full End-to-End Enterprise Flow

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Ve işte büyük orkestranın tüm enstrümanları sahnede! 60 ders boyunca öğrendiğimiz her parça tek bir kusursuz senfoniye dönüşüyor: Bilgisayarınızda Java Spring Boot kodundaki kahve mesajını değiştirip `git push` yapıyorsunuz. GitHub Actions uyanıyor, Maven ile test edip derliyor, Trivy ile tarıyor, Docker imajını GHCR'ye basıyor, GitOps reposunu güncelliyor, kümedeki Flux değişikliği çekiyor, OpenShift rotası trafiği yönlendiriyor ve siz k9s ekranında pod'ların yeşile dönmesini canlı izliyorsunuz! Tebrikler, artık bir Cloud-Native şampiyonusunuz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

AND HERE IS THE GRAND SYMPHONY! All 60 lessons unite into one breathtaking enterprise performance: You edit a single line of Java in Spring Boot and type `git push`. GitHub Actions awakens, runs Maven tests, scans with Trivy, pushes the OCI container to GHCR, commits the release SHA to the GitOps fleet repo, Flux pulls the change in-cluster, OpenShift routes live traffic, and you lean back watching the pods glow neon green in k9s! Congratulations: You are officially a Cloud-Native Master!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Uçtan uca kurumsal yaşam döngüsünün tam özeti: 1) Geliştirici Spring Boot kodunu commitleyip push eder. 2) GitHub Actions CI: `mvn test` -> Docker layered build -> Trivy security scan -> GHCR push. 3) Promotion: Actions, GitOps filosundaki `deployment.yaml` imaj etiketini günceller. 4) Flux GitOps: Değişikliği 30 saniye içinde algılar ve K8s API'sine uygular. 5) K8s/OpenShift: RollingUpdate ile yeni pod'ları ayağa kaldırır, Actuator Startup/Readiness problarını bekler, sıfır kesintiyle trafiği yeni sürüme aktarır. 6) k9s & Prometheus: Sistem mühendisi canlıda logları, CPU/RAM'i ve metrikleri izler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The definitive Cloud-Native lifecycle synthesis: 1) Dev pushes Java code. 2) GitHub Actions: Maven validation -> Layered Docker build -> Trivy CVE scan -> Push to GHCR. 3) GitOps Promotion: Automation updates fleet manifest with the Git SHA. 4) Flux Reconciliation: Polls Git, detects divergence, applies to K8s API. 5) Orchestration: Zero-downtime RollingUpdate validates Spring Boot Actuator probes before routing traffic. 6) Observability: Real-time verification via k9s HUD and Prometheus telemetry.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 🚀 BÜYÜK MEZUNİYET KONTROL LİSTESİ:
# 1. Java Kodu: Spring Boot 3.x REST API + Actuator Probes (/actuator/health)
# 2. Docker: Multi-stage build + Layered JAR + Non-Root User (UID 10001)
# 3. K8s: Deployment + ClusterIP + Ingress + HPA + Requests/Limits
# 4. OpenShift: Security Context Constraints (SCC) + Edge Route
# 5. Operators: SpringBootApp CRD + Java Operator SDK Controller
# 6. k9s: Anlık canlı log takibi ('l'), tünel ('shift-f'), kaynak izleme
# 7. FluxCD: GitRepository + Kustomization + ImageUpdateAutomation
# 8. GitHub Actions: Maven Test -> Trivy Scan -> GHCR -> GitOps Auto-Commit

# Tek Komutla Tüm Dünyayı Canlıda İzle:
k9s -n coffee-shop
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bulut bilişim yolculuğu burada bitmiyor, yeni başlıyor! Gerçek bir mühendis öğrendiği bu 8 teknolojiyi sürekli pratik yaparak, kendi projelerini konteynerize edip GitOps ile yöneterek refleks haline getirir.

🇬🇧 WATCH OUT

The Cloud-Native journey doesn't end here; it begins! Cement this mastery by building, containerizing, breaking, and healing your own microservices continuously.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- 60 derslik tam eğitim: Docker'dan K8s'e, OpenShift'ten Flux ve GitHub Actions'a.
- Tüm akış endüstri standardı Java Spring Boot mikroservis mimarisiyle uygulandı.
- Artık kurumsal bulut-yerel mimarileri baştan sona tasarlama ve yönetme yetkisine sahipsiniz!

- 60 lessons spanning Docker, Kubernetes, OpenShift, k9s, Operators, Flux, and CI/CD.
- Anchored throughout by industry-standard Java Spring Boot production implementations.
- You are fully equipped to design, deploy, and automate enterprise cloud platforms!